Selecting Reliable LLM Reasoning

DASE7506 Mini-Project 2 | Relative and joint verifiers | 31 October 2026

1. Task, question, and submitted result

The task is to choose one of four supplied solutions to a mathematics question, using the question, candidate texts, and their fixed 1,024-dimensional representations. Supervision is the final numeric answer, parsed by the unchanged course evaluator. A correct answer need not have a completely correct explanation. Training uses only the supplied training split; validation supports development and selection. [1]

Question. Can a small jointly trained verifier complement a standalone relative verifier enough to justify its cost under the CPU and asset limits? We expected candidate interaction and different outcome objectives to provide complementary rankings. The submitted method combines a relative logistic verifier with two compact neural verifiers. It uses no pretrained weights, external data, generated labels, or online inference.

Recorded metric	Result / scope
Official complete test	399/467 = 85.438972%
Official complete validation	751/798 = 94.1103% (frozen CPU implementation)
Validation disagreement subset	308/355 = 86.7606% (frozen CPU implementation)
Full CPU prediction + separate scoring	3.136154 seconds, Xeon 8368Q
Additional inference assets	11,808,974 bytes = 11.261915 MiB
Submission identity	Freeze 4ab3279f6f968f80; full SHA in manifests

Data and the scoring unit

The official splits contain 4,755 training, 798 validation, and 467 test questions. There are 2,114 training and 355 validation questions with multiple distinct valid parsed answers. Every test question is a disagreement question. The other 443 validation questions are easier unanimous-answer groups; therefore the 94.11% full-validation result must not be compared directly with the 85.44% test result. [1]

The reported submission value is the official scorer's **accuracy** field, **85.4389721627409**, on all 467 test questions. The independent evaluator reads references only after prediction. Checkpoint reproduction on the reference machine determines final eligibility; these measurements do not establish a leaderboard rank.

2. A standalone relative verifier

The first component is a logistic verifier independently fitted from scratch. Its design follows the publicly described relative-verifier combination [2]. No weights, prediction caches, datasets, or per-question test results from that repository are used. The feature implementation is shared by fitting and inference.

Representation and 32 scalar signals

```
c_i = e_i - mean_j(e_j)
r_i = c_i / (mean_j ||c_j||_2 + 1e-6)
u_i = (f_i - mean_j(f_j)) / (RMS_j(f_j - mean_j(f_j)) + .001)
x_i = concatenate(r_i, u_i) # 1,024 + 32 dimensions
```

Ten arithmetic features describe the number of checkable equalities, incorrect-equality fraction and positions, exact-equality fraction, and whether the final answer occurs on a checked left or right side. Twenty-two answer/source features describe literal numbers matching the question or previous computed values, new numbers, answer integrality, sign, size, denominator, and count/money cues. These are fallible pattern checks: literal matching does not bind objects, units, or times, and an intermediate contradiction does not imply a wrong final answer.

Each question is transformed using only its own four candidates. Column means and standard deviations are then learned from the 2,114 training disagreement groups (8,456 candidate rows); invalid endings receive negative candidate labels here. The column scale is standard deviation plus 1e-6. This per-question scaling and the subsequent fitted normalization affect training and regularization. Merely subtracting a common mean would not, by itself, change a fixed linear ranking.

Fitting and answer aggregation

```
loss = mean BCE(x_i @ w + b, y_i) + .5 * sum_d lambda_d * w_d^2
lambda_embedding = .05; lambda_scalar = .003
answer_score(a) = max_{i: answer_i=a} logit_i + .5 * log(count(a))
```

Fitting uses float64 L-BFGS, maximum 350 iterations, with an unpenalized bias. Six predeclared block-penalty combinations were compared: embedding .01/.05/.2 and scalar .003/.01. The log-count bonus 0/.5/1 was evaluated on validation. The selected combination is .05/.003 with bonus .5. This is joint fitting of all 1,056 columns, with a different prior scale for the two feature blocks. The final group score is based on raw logits, not sigmoid probabilities.

Candidates with the same parsed answer receive the same group score; any member earns the same official point. Invalid answers are excluded when a valid candidate exists. The implementation ignores IDs and does not contain an answer lookup. The reference parser is imported from the supplied code without modifications.

3. Neural comparators and the exact frozen combination

The neural components also start from random parameters. Fit-only statistics standardize the original representations; a full-dimensional ZCA transform uses eigenvalues of the training covariance with ridge .1. Both the absolute transformed vector and a within-question centered/RMS-normalized view enter the network, alongside the same 32 scalar signals. Preprocessed values are clipped to [-6,6]. No PCA truncation or validation-fitted statistic is used.

Eight members share main projection matrices, with trainable member-specific input/output scalings and independent member losses, inspired by BatchEnsemble/TabM [3]. Each member has width 128. One self-attention block compares the four candidates, includes a learned same-answer bias, and uses residual connections, layer normalization, and a 128-256-128 feedforward block. There are no candidate-position embeddings. This is a compact task adaptation, not a reproduction of official TabM benchmark results. Set attention supplies a model class for comparing candidates, not a guarantee of mathematical understanding. [4]

Training setting	Fixed recipe
Optimizer / learning rate	AdamW / .001
Weight decay / dropout	.03 / .25
Batch / maximum / patience	128 questions / 150 epochs / 20 epochs
Seeds per neural variant	17, 29, 43
Early stopping	Validation disagreement correct count; loss tie-break
Submitted neural members	Mass objective seed17; pairwise objective seed29

Two outcome objectives

```
M: mean_q,k [logsumexp_valid(s_qki) - logsumexp_correct(s_qki)]
P: mean_q,k mean_{p correct, n wrong} softplus(s_qkn - s_qkp)
Each objective also adds .1 * valid-candidate BCE.
```

A separate outcome loss is computed for each member within joint training; their logits are averaged at inference. The submitted mass-objective model selected epoch 30 (50 run); the pairwise model selected epoch 38 (58 run). Invalid candidates are masked from the neural objective. The same preprocessing is exported as NumPy arrays; inference imports neither Torch nor sklearn.

Frozen fusion; Z means valid-candidate standardization

```
S_previous = .5 * Z(L) + .5 * Z(M_seed17)
S_final = .5 * Z(S_previous) + .5 * Z(P_seed29)
Z(s)_i = (s_i - mean_valid(s)) / max(std_valid(s), 1e-6)
```

L includes its answer-group bonus; the neural group bonuses selected here are zero. The second normalization makes this a nested combination, not simply .25L+.25M+.5P. The frozen checkpoint includes every weight, transform, statistic, and configuration for all three branches. It contains no earlier Gaussian/path weights. Selection returns one Python integer per current input question.

4. Development comparisons, ablation, and scoring history

The recent model family used the same official training and validation questions for all routes. There was no additional inner holdout gate or artificial learning task. Validation was repeatedly used for early stopping and selection, so the table is adaptive development evidence, not an unbiased estimate of generalization. The neural rows are best-seed/ensemble choices from the predeclared seeds; they are not averages of independent validation replications.

Development computation	Correct /355	Interpretation
Relative logistic L	305	Standalone component
Independent candidate network	308	No candidate-attention block
Joint mass-objective network M	305	Standalone submitted neural branch
Joint pairwise network P	305	Standalone submitted neural branch
Direct answer-group CE network	308	Alternative objective, not submitted
S_previous: L + M	309	Remove final pairwise branch
S_final: S_previous + P (GPU cache)	309	Validation-tied nominated comparator
S_final: frozen CPU replay	308	Actual deployment; full validation 751/798

Removing either branch of S_previous reduces the cached development count from 309 to 305. This supports complementarity for that comparison, but does not isolate a causal benefit of reasoning-text features or attention: the independent network itself reached 308. Adding P did not improve cached validation accuracy and reduced the CPU count by one. Thus the final model is not described as a consistent validation improvement.

Numerical deployment difference

Before final test scoring, the frozen CPU validation run returned 308 disagreement answers rather than the GPU cache's 309. A validation-only comparison found a maximum raw pairwise-score difference of 3.41e-06 and combined-score difference of 6.79e-07; one near-boundary choice changed. The discrepancy was recorded and no weight, inference source, or fusion coefficient was changed to recover that point. The reported validation result uses the actual CPU run.

Frozen test records and their limits

Fixed evaluated version	Complete test
Earlier Gaussian/path baseline	385/467 = 82.4411%
S_previous	396/467 = 84.7966%
Submitted S_final	399/467 = 85.4390%

After the second finite validation comparison did not beat 309, a validation-tied non-incumbent comparator was nominated for final scoring using the existing validation/asset/name rule. Its checkpoint and source were frozen before that test. S_final gained three net correct answers on this bank, but the split disagreement and small difference do not establish a robust improvement on new questions. Earlier test totals were already known; this is iterative coursework development, not a pristine single-shot holdout study. No test-error analysis, per-question answer lookup, or parameter change after the final score was performed.

5. Resources, costs, and reproduction

Item	Recorded value
Inference asset files	All files under inference/checkpoint
Uncompressed assets	11,808,974 bytes / 16,777,216-byte limit
Fresh prediction	3.021534 seconds
Independent scoring	0.112892 seconds
Combined fresh-process wall time	3.136154 seconds / 30-second limit
CPU / observed total threads	Xeon8368Q / 1 child + 1 monitor = 2
Reference eligibility	Xeon8457C determines official timing qualification

Timing includes process startup, imports, data/asset loading, preprocessing, every branch, and the separate scorer. CUDA is hidden, each library is limited to one thread, and CPU affinity is restricted to four cores. OS thread counts are sampled. The two ZCA assets are counted separately; compression is not used to reduce the declared asset total. No dataset, references, embedding bank, or prediction cache is distributed with the release.

Recent development stage	Real fits	Stage wall seconds
Relative penalty comparison	6	21.723
Independent / joint mass networks	3 + 3	56.333 / 55.852 (concurrent)
Pairwise / answer-group objectives	3 + 3	53.267 / 80.961 (concurrent)

These 18 fits describe the recent submitted-family development stage, not the entire multi-day project. Earlier exploratory costs are retained in historical_search_cost_records.json; overlapping stage records and heterogeneous fit units are not added into a fictitious total. Each fit used one RTX 4090; two independent development jobs ran concurrently. The submission replay uses one exposed RTX 4090 and serial fits; no distributed training is used.

Clean training replay and frozen-checkpoint reproduction

The packaged entry was exercised in a new server directory using only supplied train/validation files, one RTX 4090, and no original weights as training initialization. It completed 18 serial fits in 274.072 seconds. All 10 checkpoint files and all 62 arrays were byte/exactly identical to the frozen version on this same machine. The first launcher attempt failed before any fit because the runs parent was absent; only directory creation was fixed. No new test scoring was run. This verifies this same-environment replay, not universal bitwise reproducibility across hardware. The distributed checkpoint remains the originally scored version.

```
python scripts/check_release.py --ready-to-submit
# Exact evaluation and training commands are in README.md.
```

README.md provides the exact dependency installation and single-GPU training commands. Sources, asset hashes, official scorer JSON, environment, selection history, and AI disclosure are included. The model freeze begins 4ab3279f6f968f80; its complete SHA is preserved in all evaluation and release records.

6. Interpretation, limitations, and assistance

What the comparisons support

A fast, regularized relative model already captures substantial correctness information. A randomly initialized joint model can supply complementary rankings, even when its standalone validation result is lower than an independent network. The useful empirical unit here is the full scoring computation and its cost. The results do not show that adding attention alone improves accuracy, that every extra branch helps, or that more flexible networks understand the mathematical conditions.

The remaining errors can involve missing conditions, object confusion, or requested-time mismatch while local arithmetic is consistent. The scalar extractor uses surface arithmetic and literal number provenance, so it cannot certify those semantics. Outcome labels also allow locally flawed reasoning to end in a correct number. No step-level correctness claim is made, and no new test cases were inspected to support a broader failure taxonomy.

Selection and generalization limits

Only 355 development disagreements support repeated epoch, seed, penalty, and fusion comparisons. Small selected gains can reflect this reuse. The current candidate was validation-tied in the GPU cache and one point lower on the deployed CPU; its three-answer test gain is descriptive evidence on a fixed course bank. An independent audit bank may rank the models differently. The official constraints and automated checkpoint review remain applicable. [1]

AI assistance and reuse

OpenAI Codex substantially assisted method comparison, independent implementation, training orchestration, debugging, deployment checks, packaging, and this report. The student is responsible for understanding and reviewing the implementation. No AI-generated semantic annotations were used as training labels. The relative-verifier design is credited to the public repository [2]; its published 394/467 aggregate was known, but its weights, data, test predictions, and individual test errors were not used. The supplied course IO/parser/scorer are unchanged, and GSM8K/Qwen attribution is retained in LICENSE.txt and NOTICE.txt.

References

[1] X. Wu. DASE7506 MP2: Selecting Reliable LLM Reasoning, Fall 2026. Official task/data version bb801f4f2e7629d270df765ab12c333b2dc616bb. Official guide. Accessed 31 October 2026.

[2] Anchmy. dase7506-mp2-group-relative-verifier, commit 08813bf6ad85fea5242fc40392f25dd8ba26e1cf. Method design source. Independent implementation in this release.

[3] Y. Gorishniy, A. Kotelnikov, A. Babenko. TabM: Advancing Tabular Deep Learning with Parameter-Efficient Ensembling. ICLR 2025. arXiv:2410.24210.

[4] J. Lee, Y. Lee, J. Kim, A. Kosiorek, S. Choi, Y. W. Teh. Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks. ICML 2019, PMLR 97:3744-3753. PMLR paper.

Evidence: evaluation/test-score.json, test-resources.json, validation-score.json; evidence/selection_before_test.json, validation_export_difference.json, initial_validation_selection.json, objective_validation_selection.json; CHECKPOINT_MANIFEST.json and RELEASE_MANIFEST.json. The prediction method does not read these evidence files.